

Autonomous Frequency Locking and Relocking of 689nm Diode Laser Using Machine Learning

Anurag Borah

Alkaline Research Team

May 2026

Abstract

This document describes the design, implementation, and operation of an autonomous frequency-locking and auto-relocking system for a 689 nm diode laser used in a laboratory based strontium optical lattice clock. The laser is required to be continuously maintained at a precise frequency which is referenced to an optical cavity, and must recover automatically from unlock events caused by mode hops or environmental perturbations. A rule-based approach proved insufficient because the laser's mode structure drifts on timescales of days to weeks, making static operating-point tables obsolete. We developed a machine-learning-assisted system based on a k-nearest-neighbour (kNN) consensus predictor trained on historical two-dimensional mode-structure scans. The predictor identifies mode-hop-free operating regions and selects optimal (current, piezo voltage) candidates that allow the wavemeter PID controller to maintain the target frequency. A slow current feedback loop keeps the piezo voltage centred within the mode-hop-free plateau, reducing the frequency of mode-hop events. The system has demonstrated continuous locked operation over multi-day periods with fully autonomous relock after interruptions.

Contents

1	Introduction	1
1.1	Document Structure	1
2	Hardware Setup	2
2.1	689 nm Diode Laser	2
2.2	HighFinesse Wavemeter	2
2.3	Wavemeter Calibration Reference	3
3	Laser Mode Structure	3
3.1	Longitudinal Modes and Mode Hops	3
3.2	Mode-Hop-Free Regions	3
3.3	Two-Dimensional Mode Mapping	4
3.3.1	Piezo-Axis Scan: mode_search.py	4
3.3.2	Current-Axis Scan: current_search.py	4
4	Machine Learning Mode Finder	5
4.1	Motivation	5
4.2	Data Ingestion: train.py	5
4.3	kNN Consensus Map: ConsensusMap	6
4.3.1	Temporal Filtering	6
4.3.2	Normalised Distance Metric	6
4.3.3	Prediction and Stability Criterion	6
4.4	Candidate Identification: find()	6
5	Autonomous Locking System: lock_v2.py	7
5.1	Architecture	7
5.2	CSV Logging	7
5.3	Candidate List Management	8
5.4	Frequency Acquisition	8
5.4.1	Setting the Operating Point	8
5.4.2	Adaptive Piezo Sweep	9
5.4.3	Stability Check and Lock Engagement	9
5.5	Continuous Monitoring	9
5.6	Unlock Detection and Recovery	10
5.6.1	Classification of Unlock Events	10
5.6.2	Type 1 — Normal Unlock (Drift)	10
5.6.3	Type 2 — Mode Hop	10
5.6.4	Candidate List Exhaustion	10
6	Piezo Centering: lock_v3.py	11
6.1	Motivation	11
6.2	Physical Principle	11
6.3	The $I^*(V)$ Curve	11
6.4	Centering Algorithm	12
7	Wavemeter Drift	13
8	System Performance	13

9	Operational Procedures	14
9.1	Initial Setup and Training	14
9.2	Starting the Lock	14
9.3	Periodic Maintenance	15
10	Future Work	15
11	Appendix: Key Parameters	16

1 Introduction

Laser frequency stabilisation is a fundamental requirement in precision atomic physics experiments. For strontium spectroscopy and laser cooling on the intercombination line, the 689 nm laser must be held within a few megahertz of the atomic resonance continuously, often for periods of days. External cavity diode lasers (ECDLs) in the Littrow configuration are widely used for this purpose, but they are susceptible to two distinct classes of instability: slow frequency drift driven by thermal and mechanical perturbations, and sudden discontinuous frequency jumps — mode hops — caused by mode competition within the laser cavity.

Slow drift is routinely corrected by a proportional-integral-derivative (PID) controller that feeds back onto the piezoelectric transducer (piezo) which adjusts the grating angle and hence the cavity length. The HighFinesse wavemeter used in this work provides an absolute frequency measurement accurate to a few megahertz, which serves as the error signal for this PID. Mode hops, however, cannot be corrected by the PID alone: they cause the lasing mode to jump to a different longitudinal mode cavity resonance, producing a frequency discontinuity of several gigahertz — far outside the PID capture range.

Early relock scripts addressed mode hops by performing a blind sweep of the piezo voltage across the accessible range, searching for a voltage at which the frequency coincidentally fell near the target. This approach is unreliable for two reasons. First, the laser may have hopped to a mode that cannot reach the target frequency at any accessible piezo voltage with the current diode current — changing the diode current is required. Second, the set of (current, piezo) combinations that produce mode-hop-free operation at the target frequency drifts over days to weeks as the laser ages and thermal conditions change, making any fixed operating-point table stale.

This work replaces the blind-sweep approach with a system that learns the laser’s mode structure from historical scans and uses this knowledge to predict optimal operating points for each lock attempt. The system automatically adapts to drift in the mode structure by weighting recent data more heavily. Additionally, a slow outer feedback loop uses the learned structure to keep the piezo centred within the mode-hop-free plateau during locked operation, proactively reducing the rate of mode-hop events.

1.1 Document Structure

Section 2 describes the hardware setup. Section 3 explains the laser’s mode structure and how it is characterised by two-dimensional scans. Section 4 describes the machine learning model used to predict operating points. Section 5 covers the locking system architecture, acquisition procedure, and relock logic. Section 6 describes the piezo centering extension. Section 7 discusses wavemeter drift and its mitigation. Section 8 summarises performance, and Section 9 outlines future work.

2 Hardware Setup

2.1 689 nm Diode Laser

The laser is a Toptica DLC Pro housing an external cavity diode laser operating at 689 nm, corresponding to the $^1S_0 \rightarrow ^3P_1$ intercombination transition of strontium-88 at 434.829 THz. The DLC Pro provides two primary actuators for frequency control:

Piezo voltage acts on the grating angle of the Littrow cavity via a piezoelectric transducer, changing the cavity length and hence the lasing frequency. The piezo responds in approximately 1 ms and is the fast actuator used by the PID controller. The accessible range is 0–150 V; in practice the system operates between 20 V and 60 V.

Diode current sets the gain medium carrier density, which affects the laser frequency through the refractive index of the gain chip. The current responds in approximately 100 ms and is used as the slow actuator in this work. The operating range is 89.5–92.0 mA. Below 89.5 mA the laser intensity is insufficient for reliable wavemeter readings.

The laser is connected to a control computer running the Toptica SDK (`lasersdk.dlcpro.v2_0_3`) over TCP at IP address 172.29.13.247, port 1998. The SDK exposes the following control nodes:

SDK path	Description
<code>laser1.dl.cc.current_set</code>	Set diode current (mA)
<code>laser1.dl.cc.current_act</code>	Read actual diode current (mA)
<code>laser1.dl.pc.voltage_set</code>	Set piezo voltage (V)
<code>laser1.dl.pc.voltage_act</code>	Read actual piezo voltage (V)
<code>laser1.dl.lock.lock_enabled</code>	Enable/disable wavemeter PID lock
<code>laser1.dl.lock.state_txt</code>	Current lock status (string)
<code>laser1.dl.tc.temp_act</code>	Read diode temperature (°C)

Table 1: Toptica DLC Pro SDK control nodes used in this work.

2.2 HighFinesse Wavemeter

The frequency reference is a HighFinesse WS-U wavemeter, which provides absolute frequency measurements via a multi-channel Fizeau interferometer array. The wavemeter is interfaced through the shared library `libwlmData.so` (Linux) using the vendor-supplied Python wrapper `wlmData.py`. The 689 nm laser is connected to channel 3.

The primary call used to read frequency is:

```
freq_THz = wlmData.dll.GetFrequencyNum(channel=3, 0.0)
```

A return value ≤ 0 indicates no valid signal (beam blocked, out of range, or wavemeter not ready) and is treated as a missing sample.

The wavemeter hosts an internal PID controller that can directly actuate the Toptica piezo over the SDK connection. When `lock_enabled = True`, the wavemeter PID continuously adjusts the piezo voltage to minimise the error between the measured frequency and the configured setpoint. The lock window used throughout this work is:

$$f_{\text{target}} = 434.829035 \text{ THz}, \quad \Delta f = 3 \text{ MHz}$$

$$f_{\text{min}} = f_{\text{target}} - \Delta f, \quad f_{\text{max}} = f_{\text{target}} + \Delta f$$

2.3 Wavemeter Calibration Reference

The wavemeter is calibrated against an ultrastable reference laser connected to a dedicated channel. When this reference laser is locked and stable, the wavemeter’s frequency readings are accurate to within the instrument specification. When the reference laser unlocks — which occurs periodically — the wavemeter loses its calibration anchor and can drift by several megahertz over hours. This drift is discussed further in Section 7.

3 Laser Mode Structure

3.1 Longitudinal Modes and Mode Hops

An external cavity diode laser operates on a single longitudinal cavity mode selected by the Littrow grating angle. The free spectral range of a typical external cavity is on the order of several gigahertz. As the piezo voltage changes the cavity length, the resonance frequency of the selected mode shifts continuously (mode-hop-free tuning). However, when the gain curve and the cavity resonance become sufficiently misaligned, the laser can abruptly jump to an adjacent longitudinal mode — a mode hop. This jump is discontinuous and typically of order 5–20 GHz.

A mode hop renders the PID lock ineffective: the frequency error suddenly becomes comparable to the free spectral range, which is far outside the PID integrator’s capture range. The lock circuit either saturates or the wavemeter simply reports a frequency far from the setpoint, and the laser remains on the wrong mode indefinitely unless the locking software intervenes.

3.2 Mode-Hop-Free Regions

At any fixed diode current, there exists a range of piezo voltages over which the laser tunes continuously without mode hopping. This is called a **mode-hop-free (MHF) region**. The width of such a region in the piezo axis is typically 2–10 V for the laser in this work, corresponding to a continuous tuning range of tens to hundreds of megahertz.

The MHF region boundaries, and the relationship between piezo voltage and laser frequency within a region, both depend on the diode current. At different current values,

different modes are selected and the MHF region positions shift. This gives rise to a two-dimensional (current, piezo) landscape of mode structure, which must be mapped to find operating points where the target frequency can be reached within a mode-hop-free region.

3.3 Two-Dimensional Mode Mapping

3.3.1 Piezo-Axis Scan: `mode_search.py`

The script `mode_search.py` performs a systematic two-dimensional scan to map the laser mode structure along the piezo axis. The scan geometry is:

- **Outer loop:** diode current swept from 89.0 to 92.0 mA in 0.1 mA steps
- **Inner loop:** piezo voltage swept from 20.0 to 60.0 V in 0.1 V steps

At each (current, piezo) point the script waits 200 ms for the laser to stabilise, then records the wavemeter frequency. The result is a grid of frequency measurements over the full (current, piezo) parameter space, totalling approximately $31 \times 401 = 12,431$ points per scan, requiring approximately 40 minutes.

MHF region detection on the piezo axis: for each fixed current row, the script walks along the piezo axis and identifies contiguous segments where adjacent frequency values differ by less than a mode-hop threshold of 1 GHz. These are the mode-hop-free regions for that current. For each MHF region that spans the target frequency, it interpolates the exact piezo voltage at which the frequency crosses f_{target} and records a candidate operating point:

Field	Description
<code>current_ma</code>	Diode current at which this candidate was found
<code>piezo_v</code>	Interpolated piezo voltage where $f = f_{\text{target}}$
<code>mhf_piezo_min_v</code>	Lower bound of the MHF region (V)
<code>mhf_piezo_max_v</code>	Upper bound of the MHF region (V)
<code>mhf_width_v</code>	Width of the MHF region (= max – min)

Table 2: Fields recorded per candidate in a `mode_search` scan.

Each scan is saved as a timestamped CSV (raw data), JSON (candidate list), and PNG (2D frequency map). Scans are accumulated in `mode_search/` and used as training data for the kNN model described in Section 4.

3.3.2 Current-Axis Scan: `current_search.py`

The script `current_search.py` performs the complementary scan: the outer loop steps through piezo voltages, and the inner loop sweeps diode current at each fixed piezo. This maps the mode structure along the **current axis** at each piezo value.

- **Outer loop:** piezo voltage from 30.0 to 60.0 V in 0.5 V steps
- **Inner loop:** current from 89.0 to 92.0 mA in 0.1 mA steps

MHF detection on the current axis: for each fixed piezo row, contiguous current segments with inter-step frequency jumps below 1 GHz are identified. For each segment crossing the target frequency, the script interpolates the exact current $I^*(V)$ at which $f = f_{\text{target}}$ and records:

Field	Description
piezo_v	Fixed piezo voltage at which this candidate was found
current_mA	Interpolated current $I^*(V)$ where $f = f_{\text{target}}$
mhf_current_min_mA	Lower bound of the current-axis MHF region
mhf_current_max_mA	Upper bound of the current-axis MHF region
mhf_width_mA	Width of the current MHF region

Table 3: Fields recorded per candidate in a current_search scan.

The set of points $\{(V_i, I^*(V_i))\}$ across all piezo values defines a sparse curve $I^*(V)$ — the function mapping piezo voltage to the current required to hit the target frequency. This curve is the basis of the piezo centering algorithm in Section 6.

4 Machine Learning Mode Finder

4.1 Motivation

A single mode_search scan provides a snapshot of the laser’s mode structure on a particular day. However, the structure drifts over time: MHF region boundaries can shift by more than 1 V between weekly scans, and the (current, piezo) combinations that reach the target frequency change accordingly. Simply using the most recent scan for prediction is brittle — a single scan may contain noise or wavemeter glitches. Using all historical scans with equal weight is worse, as old data may be systematically misleading.

A further complication is that the (current, piezo) $\rightarrow$ frequency map is **multi-valued** near mode-hop boundaries: depending on which mode the laser happens to be on when a point is measured, the frequency can be either of two values separated by several gigahertz. A parametric model such as a multi-layer perceptron would average over these values, producing predictions in between that correspond to no physical operating mode. For this reason, a non-parametric approach that preserves the multi-modality of the data is required.

4.2 Data Ingestion: train.py

The script `ml_mode_finder/train.py` reads all `mode_search/search_*.csv` files and concatenates them into a single archive `checkpoints/scan_index.npz`. Each row in the archive records (current_mA, piezo_v, frequency_THz, scan_id). Samples with frequency readings outside the physical plausible range (434.5–435.0 THz) are discarded as wavemeter glitches. The scan ID is an integer index assigned to each source file in chronological order.

4.3 kNN Consensus Map: ConsensusMap

The class `ConsensusMap` in `find_mode.py` implements a k-nearest-neighbour predictor over the archived `mode_search` data. For a query point (I_q, V_q) , it returns a median frequency estimate and a stability flag.

4.3.1 Temporal Filtering

To track laser drift, only the most recent $N = 5$ scans are used (`RECENT_SCANS`). Data from older scans is excluded from the kNN tree. This means the model always reflects the laser’s current operating regime rather than an average over its entire lifetime.

4.3.2 Normalised Distance Metric

Diode current and piezo voltage have different physical units and different sensitivities. A 1 mA change in current and a 1 V change in piezo voltage affect the frequency by different amounts and are not directly comparable. To make kNN distances meaningful, both axes are normalised to $[0, 1]$ before constructing the search tree:

$$\tilde{I} = \frac{I - I_{\min}}{I_{\max} - I_{\min}}, \quad \tilde{V} = \frac{V - V_{\min}}{V_{\max} - V_{\min}}$$

where $I_{\min} = 89.5$ mA, $I_{\max} = 92.0$ mA, $V_{\min} = 20.0$ V, $V_{\max} = 60.0$ V. The kNN tree (`scipy.cKDTree`) is built over the normalised two-dimensional coordinates $(\tilde{I}, \tilde{V})$.

4.3.3 Prediction and Stability Criterion

For a query point (I_q, V_q) , the $k = 20$ nearest neighbours are retrieved. Let $\{f_1, f_2, \dots, f_k\}$ be their recorded frequencies. The prediction is:

$$\hat{f} = \text{median}(\{f_1, \dots, f_k\})$$

The stability flag is defined by the interquartile range (IQR):

$$\text{IQR} = Q_3 - Q_1, \quad \text{stable} = [\text{IQR} < 1.5 \text{ GHz}]$$

The IQR threshold of 1.5 GHz is chosen to be smaller than the typical mode-hop gap (5–20 GHz) but larger than the thermal drift of MHF region boundaries across recent scans (1 GHz). A query inside a stable MHF plateau returns `stable = True`; a query near a mode-hop boundary, where different historical scans recorded different modes, returns `stable = False` due to the large spread among neighbours.

This is the key advantage of kNN over parametric models: the multi-modality at boundaries is preserved as high variance rather than averaged away. The IQR thus serves as an intrinsic measure of prediction reliability.

4.4 Candidate Identification: `find()`

The function `find()` constructs a grid over the full (current, piezo) parameter space with steps of $0.1 \text{ mA} \times 0.1 \text{ V}$ and queries `ConsensusMap.predict()` at every grid point, obtaining a predicted frequency and stability flag for each.

The candidate identification algorithm then operates row by row (fixed current, varying piezo):

1. **Identify stable, smooth segments:** walk along the piezo axis. Accumulate a run of consecutive grid cells where (a) the cell is flagged stable, and (b) the frequency difference from the previous cell is below the mode-hop threshold of 1 GHz. Break the run whenever either condition fails.
2. **Filter by MHF width:** segments narrower than 2.0 V are discarded (likely artefacts). Segments wider than 15.0 V are also discarded — the kNN model can produce artificially wide “regions” by smoothing across multiple true modes when data density is low; the 15 V upper limit removes these artefacts.
3. **Check frequency crossing:** retain only segments for which $f_{\min(\text{segment})} \leq f_{\text{target}} \leq f_{\max(\text{segment})}$.
4. **Interpolate operating point:** within the qualifying segment, find the piezo voltage V^* at which $\hat{f}(I, V^*) = f_{\text{target}}$ by linear interpolation between adjacent grid cells.
5. **Record candidate:** store $(I, V^*, V_{\text{MHF},\min}, V_{\text{MHF},\max})$.

All candidates are sorted by (current ascending, piezo ascending). The preference for lower current reflects that higher drive current increases thermal load and reduces laser lifetime. The sorted list is returned for use by the locking system.

5 Autonomous Locking System: lock_v2.py

5.1 Architecture

The locking script runs as two concurrent threads:

Thread	Role
Worker thread	All hardware I/O: reads wavemeter, controls laser, writes CSV. Runs the acquisition, monitor, and relock logic.
Main thread	Runs <code>matplotlib</code> animation for live display. Blocks on <code>plt.show()</code> until the window is closed.

Table 4: Thread responsibilities in lock_v2.py.

Data flows from the worker thread to the main thread via `_buf`, a dictionary of `collections.deque` objects. Under Python’s Global Interpreter Lock (GIL), `deque` append and pop operations are atomic, so no explicit synchronisation is required. The deques hold the time series of frequency offset, piezo voltage, and diode current for live plotting.

5.2 CSV Logging

Every hardware sample is written to a timestamped CSV file at `relock_log/ml_lock_ch3_YYYYMMDD_HHMMSS.csv`. The columns are:

Column	Description
timestamp_utc	UTC timestamp, millisecond precision
channel	Wavemeter channel (3)
frequency_THz	Measured frequency (THz, 6 decimal places)
piezo_V	Piezo voltage reading (V)
current_mA	Diode current reading (mA)
temp_C	Diode temperature (°C)
lock_state	Lock status string from DLC Pro SDK
in_window	1 if $f_{\min} \leq f \leq f_{\max}$, else 0

Table 5: CSV log columns recorded at each 200 ms sample.

The CSV is flushed after every row so that data is preserved even if the script terminates abnormally.

5.3 Candidate List Management

Calling `find()` requires querying the kNN tree at every point of the (current, piezo) grid, which takes several seconds. To avoid this overhead on every relock event, the candidate list is computed once at startup and stored in memory. The structure is:

```
state = {'cands': [], 'cand_idx': 0}
```

`_fetch()` calls `find()` and resets the list index to 0. `_acquire(reset)` iterates through candidates starting from `cand_idx`. If `reset=True`, it restarts from candidate 1 without calling `find()` again. `find()` is only called again when the full candidate list has been exhausted without a successful lock, ensuring that frequent relocks do not repeatedly pay the prediction cost.

5.4 Frequency Acquisition

5.4.1 Setting the Operating Point

For each candidate $(I_c, V_c, V_{\text{MHF},\min}, V_{\text{MHF},\max})$, the function `try_candidate()` performs:

1. Disable the wavemeter PID lock if currently active
2. Set current to I_c via `current_set`
3. Set piezo to V_c via `voltage_set`
4. Poll `voltage_act` until the piezo settles within 0.5 V of V_c , or time out after 10 s
5. Wait an additional 300 ms for the laser to thermally settle

After settling, the wavemeter is read. A headroom check is performed: if there is less than 1.0 V of piezo travel between the current position and the relevant MHF boundary in the

direction of the frequency offset, the candidate is skipped (insufficient room to sweep to target).

5.4.2 Adaptive Piezo Sweep

`piezo_sweep_and_lock()` sweeps the piezo voltage toward the target frequency using two step sizes:

$$\delta V = \begin{cases} 100 \text{ mV} & \text{if } |f - f_{\text{target}}| > 50 \text{ MHz} \quad (\text{coarse}) \\ 10 \text{ mV} & \text{otherwise} \quad (\text{fine}) \end{cases}$$

The coarse/fine threshold of 50 MHz ensures rapid approach from far away while providing resolution near the target for clean window entry. At each step the piezo is incremented, the system waits 200 ms, and the wavemeter is read. The sweep continues until the frequency enters the window $[f_{\min}, f_{\max}]$.

MHF boundary enforcement: if the next step would move the piezo outside the predicted MHF region $[V_{\text{MHF,min}}, V_{\text{MHF,max}}]$, the sweep aborts and the candidate is declared failed. This prevents walking across a mode-hop boundary during acquisition.

Voltage safety: a hardware safety check is enforced before every piezo command. If the requested voltage is outside the absolute range [20 V, 60 V], the script terminates immediately via `os._exit(1)` to prevent hardware damage.

5.4.3 Stability Check and Lock Engagement

Once the frequency has entered the window, a stability check is performed: the wavemeter is polled up to 10 times (200 ms apart) looking for 5 consecutive in-window readings. This confirms that the laser is genuinely on the correct mode and not merely crossing the window transiently during a mode hop. If 5 consecutive readings are not achieved, the candidate is declared failed.

On passing the stability check, the wavemeter PID lock is engaged:

```
lock.lock_enabled.set(True)
time.sleep(SETTLE_AFTER_LOCK)    # 5 s
```

After the settle time, the frequency is read again and the lock state is queried. If the frequency is inside the window and the lock is confirmed active, acquisition is declared successful.

5.5 Continuous Monitoring

After a successful lock, the worker thread enters a monitor loop that runs indefinitely at 200 ms intervals. Each iteration:

1. Reads the wavemeter frequency
2. Reads piezo voltage, diode current, temperature, and lock state from the DLC Pro
3. Appends all values to the CSV log and to the shared plot buffers
4. Checks whether the laser is still locked (`in_win = True` and `lock_on = True`)
5. If locked: resets the relock attempt counter, updates status to “Locked”

6. If unlocked: determines the unlock type and responds accordingly

5.6 Unlock Detection and Recovery

5.6.1 Classification of Unlock Events

Two quantities are evaluated at each monitor step to classify the unlock:

MHF membership: is the current piezo voltage within the MHF bounds of the locked operating point?

$$\text{in_MHF} = V_{\text{MHF,min}} \leq V_{\text{piezo}} \leq V_{\text{MHF,max}}$$

Large mode hop: is the frequency offset from target larger than 5 GHz?

$$\text{large_hop} = |f - f_{\text{target}}| > 5 \text{ GHz}$$

5.6.2 Type 1 — Normal Unlock (Drift)

Condition: `in_MHF = True` and `large_hop = False`

The laser has drifted out of the ± 3 MHz lock window, but the piezo is still within the MHF region — the laser is still on the correct longitudinal mode. This is a normal PID capture failure due to a large frequency excursion.

Response: bounded piezo sweep within the known MHF bounds. The PID lock is disabled, the sweep procedure from Section 5.3.2 is run, and the PID is re-engaged. Up to `MAX_RELOCK_ATTEMPTS = 3` attempts are made before escalating.

5.6.3 Type 2 — Mode Hop

Condition: `in_MHF = False`, or `large_hop = True`, or Type 1 has exhausted 3 attempts

The laser has jumped to a different longitudinal mode. The MHF bounds of the current operating point are no longer valid.

Response: restart from candidate 1 in the stored candidate list (reset index without calling `find()` again). The `_acquire()` procedure is re-run from the beginning of the list.

The large-hop guard (`large_hop = True`) short-circuits the MHF check: if the frequency offset is greater than 5 GHz, a bounded piezo sweep within a few volts is clearly futile regardless of MHF membership, and the system immediately proceeds to a full re-acquisition.

5.6.4 Candidate List Exhaustion

If every candidate in the stored list fails to achieve lock, `find()` is called again to fetch a fresh prediction. This handles the case where the laser mode structure has changed substantially since the last prediction.

6 Piezo Centering: `lock_v3.py`

6.1 Motivation

During locked operation, the wavemeter PID adjusts the piezo voltage to maintain the target frequency as the laser ages and the environment changes. This causes the piezo voltage to drift slowly over time. If it drifts toward the edge of the MHF plateau and crosses the boundary, a mode hop occurs — a Type 2 unlock event that requires a full re-acquisition.

This class of event is avoidable. If the piezo could be kept near the **centre** of the MHF plateau rather than near the edge, there would be more headroom to absorb slow drift before a mode-hop boundary is reached. The centering loop in `lock_v3.py` achieves this by slowly adjusting the diode current.

6.2 Physical Principle

The wavemeter PID enforces the constraint:

$$f(I, V) = f_{\text{target}}$$

at all times during locked operation. This defines a curve in the (I, V) plane — the locked operating manifold. Along this curve, the PID moves the piezo V to compensate for any perturbation to f . If the current I is changed, the frequency $f(I, V)$ shifts, and the PID responds by moving the piezo to a new V that restores $f = f_{\text{target}}$. The net effect of a current change δI is a piezo displacement:

$$\delta V = -\frac{\frac{\partial f}{\partial I}}{\frac{\partial f}{\partial V}} \cdot \delta I$$

Both partial derivatives are negative for the laser in this work (increasing current lowers frequency; increasing piezo voltage also lowers frequency), so their ratio is positive: a positive δI produces a positive δV . This means increasing the current causes the PID to drive the piezo upward, and vice versa.

By choosing the sign and magnitude of δI appropriately, the equilibrium piezo position can be steered to the centre of the MHF plateau.

6.3 The $I^*(V)$ Curve

The function $I^*(V)$ — the current at which the target frequency is hit at piezo voltage V — is obtained directly from the `current_search` scans. Recall that each `current_search` JSON contains `all_candidates`, a list of $(V_i, I^*(V_i))$ pairs from that scan. `CurrentSearchMap` loads the three most recent such JSONs, groups candidates by piezo voltage, takes the median current across scans at each voltage, and stores the result as two arrays `piezo_vals` and `current_vals`. Interpolation is performed with `numpy.interp`.

Using the median across multiple recent scans makes the $I^*(V)$ curve more robust to noise in individual scans. The three-scan window is consistent with the temporal filtering used in the kNN model.

If a query voltage falls outside the range of available data, `get_target_current()` returns `None` and centering is skipped for that iteration.

6.4 Centering Algorithm

The centering logic executes within the worker thread every `CENTERING_INTERVAL_S = 30` seconds, but only after the laser has been continuously locked for at least `CENTERING_MIN_LOCK_S = 60` seconds. The delay after locking prevents the centering loop from interfering with a freshly acquired lock that has not yet settled.

At each centering check:

1. Compute the MHF plateau centre:

$$V_{\text{centre}} = \frac{V_{\text{MHF,min}} + V_{\text{MHF,max}}}{2}$$

2. Compute the piezo error:

$$\varepsilon = V_{\text{piezo,act}} - V_{\text{centre}}$$

3. If $|\varepsilon| \leq 1.5$ V (`CENTERING_DEADBAND_V`), no action is taken.
4. Otherwise, look up the target current at the plateau centre:

$$I_{\text{target}} = I^*(V_{\text{centre}})$$

5. Compute and apply the current nudge:

$$\delta I = \text{clip}(0.5 \cdot (I_{\text{target}} - I_{\text{act}}), \pm 0.15 \text{ mA})$$

$$I_{\text{new}} = \text{clip}(I_{\text{act}} + \delta I, I_{\text{min}}, I_{\text{max}})$$

The gain of 0.5 (50% of the full correction per step) prevents overshoot. The ± 0.15 mA hard cap prevents large sudden current changes that could disturb the lock. After the nudge, the wavemeter PID naturally moves the piezo toward V_{centre} over the next few seconds.

The `lock_acquired_at` timestamp is reset on every relock event, ensuring that centering never fires immediately after a mode-hop recovery when the operating point may not yet be stable.

Parameter	Value	Description
CENTERING_INTERVAL_S	30 s	Time between centering checks
CENTERING_MIN_LOCK_S	60 s	Minimum continuous lock before centering activates
CENTERING_DEADBAND_V	1.5 V	Minimum piezo error to trigger a nudge
CENTERING_GAIN	0.5	Fraction of full correction per step
CENTERING_MAX_NUDGE_MA	0.15 mA	Hard cap on current change per nudge
RECENT_SCANS_CS	3	Number of recent current_search scans used

Table 6: Piezo centering parameters in lock_v3.py.

7 Wavemeter Drift

The HighFinesse wavemeter is calibrated against an ultrastable reference laser. When this reference laser is locked and stable, the wavemeter’s absolute accuracy is within its specification. When the reference laser unlocks, the wavemeter loses its calibration anchor.

Over a weekend without a stable reference, the wavemeter frequency readings were observed to drift by approximately 5 MHz. This was manifested as the 689 nm lock system holding the laser at what it perceived to be 434.829040 THz, while the true frequency (relative to the atomic transition) was 434.829035 THz. The symptom was that the laser could be held within the wavemeter’s lock window but was off-resonance with the atoms.

Current mitigation: `FREQ_TARGET` is updated manually in `find_mode.py` and `current_search.py` when drift is observed. The current target value is 434.829035 THz.

Planned mitigation: a monitoring routine will watch the reference laser’s wavemeter channel. When the reference laser returns to within a tolerance of its known frequency, `wlmData.dll.Calibration()` will be called automatically to recalibrate the wavemeter. This will eliminate the need for manual target updates.

8 System Performance

The locking system has been run continuously for multi-day periods. The key metrics observed are:

- **Lock acquisition time:** typically 15–60 s from script start, depending on how far the laser has drifted from the predicted operating point.
- **Type 1 relock time:** 5–15 s for a bounded piezo sweep followed by PID re-engagement.
- **Type 2 relock time:** 30–90 s for a full re-acquisition through the candidate list, including current and piezo repositioning and sweep.

- **Mode-hop rate:** qualitatively reduced after introduction of the piezo centering loop in v3, as the piezo is kept farther from MHF boundaries.

CSV logs are analysed post-hoc using `plot_relock_log.py`, which plots frequency offset, piezo voltage, diode current, and temperature as a function of time. Known-bad time segments (e.g. manual interventions, reference laser outages) can be masked by specifying time ranges in the REMOVE list.

9 Operational Procedures

9.1 Initial Setup and Training

Before running the lock script for the first time, or after a major change to the laser (component replacement, significant realignment), the mode structure must be mapped and the model retrained:

1. Run a piezo-axis scan:

```
python mode_search.py
```

2. Run a current-axis scan:

```
python current_search.py
```

3. Retrain the kNN model:

```
python -m ml_mode_finder.train
```

9.2 Starting the Lock

```
python3 -m ml_mode_finder.lock_v3
```

At startup the script prints:

- Wavemeter version and connection status
- [CENTER] $I^*(V)$ curve: confirming how many `current_search` scans were loaded and the range of the $I^*(V)$ curve
- kNN model summary: number of scan points and scans used
- Predicted candidates with their (current, piezo, MHF bounds)

9.3 Periodic Maintenance

Task	Frequency	Command
New mode_search scan	Weekly	python mode_search.py
New current_search scan	After mode_search	python current_search.py
Retrain kNN model	After new mode_search	python - m ml_mode_finder.train
Update FREQ_TARGET	When wavemeter drifts	Edit find_mode.py

Table 7: Recommended periodic maintenance schedule.

10 Future Work

Automated wavemeter recalibration: implement the monitoring routine described in Section 7 to detect reference laser lock recovery and trigger wavemeter recalibration automatically, eliminating manual FREQ_TARGET updates.

Online model update: retrain the kNN model after each lock session using the CSV log data, which implicitly contains the mode structure (the sequence of piezo voltages visited during acquisition and monitoring). This would make the model self-updating without requiring scheduled manual scans.

Adaptive centering gain: the centering gain is currently fixed at 0.5. An adaptive gain that increases when the piezo is close to the MHF boundary and decreases when it is near the centre would provide faster correction while maintaining stability.

Reference laser co-locking: if the reference laser that calibrates the wavemeter can be made more robust (or a backup reference provided), the wavemeter drift issue would be resolved at the source rather than mitigated in software.

11 Appendix: Key Parameters

Parameter	Value	Description
FREQ_TARGET	434.829035 THz	Target laser frequency
FREQ_WINDOW	± 3 MHz	Lock acceptance window
I_MIN / I_MAX	89.5 / 92.0 mA	Current operating range
V_MIN / V_MAX	20.0 / 60.0 V	Piezo operating range
DELAY	0.2 s	Wavemeter poll interval
SETTLE_AFTER_SET	0.3 s	Settle time after current/piezo set
SETTLE_AFTER_LOCK	5.0 s	Settle time after PID engagement
STEP_COARSE_V	0.1 V (100 mV)	Piezo sweep step far from target
STEP_FINE_V	0.01 V (10 mV)	Piezo sweep step near target
COARSE_THRESH_MHZ	50 MHz	Switch from coarse to fine step
STABLE_READINGS	5	Consecutive in-window reads for stability
MAX_RELOCK_ATTEMPTS	3	Type 1 relock attempts before escalation
LARGE_HOP_THZ	0.005 THz (5 GHz)	Mode-hop guard threshold
RECENT_SCANS	5	kNN: recent scans used
K_NEIGHBORS	20	kNN: number of neighbours
STABILITY_IQR_THZ	0.0015 THz (1.5 GHz)	kNN: stability IQR threshold
GRID_CURRENT_STEP	0.1 mA	Candidate search grid step (current)
GRID_PIEZO_STEP	0.1 V	Candidate search grid step (piezo)
MIN_MHF_WIDTH_V	2.0 V	Minimum MHF width to accept
MAX_MHF_WIDTH_V	15.0 V	Maximum MHF width (artefact filter)

Table 8: Complete parameter reference for all system components.